

CAF — Mimic & Creative Intelligence

Aggregated export — 2026-07-10. Source markdown in repo is unchanged.

Bundle id: `07-caf-mimic-and-creative-intelligence`

Included: docs/MIMIC_FLOWS_COMPLETE_GUIDE.md

CAF Core — Top-Performer Mimic Flows (Complete Guide)

Version: CAF Core repository snapshot — **updated July 2026** (see `docs/CAF_CURRENT_STATE_CONTEXT_PACK.md` for repo-derived truth)

Audience: Operators, engineers, and AI assistants (e.g. ChatGPT project folders)

Scope: Top-performer mimic family, **New Visual** (`FLOW_VISUAL_FIRST_CAROUSEL`), **Why Mimic** (`FLOW_WHY_MIMIC_CAROUSEL`), BVS, and upstream/downstream dependencies

***Updated current-state note (2026-07):** Visual-first is **not** competitor-frame replication — it uses `execution_mode: "new_visual"` with idea + BVS and empty `reference_items`. Why Mimic is a separate SIL-driven lane. BVS (`brand_bibles` → `bvs_v1`) is production. Prefer `docs/CAF_CURRENT_STATE_CONTEXT_PACK.md` when this guide conflicts with source.*

1. Executive summary

Mimic flows are optional CAF pipeline lanes that recreate the **visual pattern** of archived **top-performer** social posts while generating **fresh copy** aligned to the brand and the current signal-pack idea. They are **not** pixel-perfect clones: logos, faces, and copyrighted imagery are explicitly excluded (`twist_brief.visual_only`).

The system splits work into two phases:

1. **Generate (copy)** — OpenAI LLM writes caption, hashtags, and slide text faithful to the reference *structure* (hook device, pacing, CTA shape).
2. **Render (visual)** — Image models (BFL FLUX, DashScope Qwen, NVIDIA NIM, or OpenAI `gpt-image-1`) produce plates; Puppeteer Handlebars templates or Sharp compositing overlay the LLM copy with typography derived from upstream vision analysis (Nemotron / Document AI).

Mimic flows sit in the standard CAF funnel:

Signal Pack → Candidates → Decision Engine → Content Job → Draft → Render
→ Review → Publish

Each mimic job is keyed by `task_id` like any other content job. Render metadata lives on `generation_payload.mimic_v1` (source of truth for render). Carousel mimic jobs additionally snapshot `draft_package_snapshot` with `package_type`: `"mimic_carousel_package"` — this is **distinct** from normal `FLOW_CAROUSEL` / `carousel_package`.

2. Flow types

Flow type	Platform format	Reference / lane	When to use
FLOW_TOP_PERFORMER_MIMIC_IMAGE	Single post (post)	top_performer_deep	Idea grounded to a top performer with exactly one archived reference frame
FLOW_TOP_PERFORMER_MIMIC_CAROUSEL	Carousel	top_performer_carousel	Manual mimic picks (Mimic · Carousel tab) with 2+ archived frames; execution_mode: classic
FLOW_VISUAL_FIRST_CAROUSEL	Carousel	New visual (no TP frames)	Ideas-from-insights with carousel_style: visual_first OR mixed — execution_mode: new_visual, idea + BVS driven
FLOW_WHY_MIMIC_CAROUSEL	Carousel	TP references + SIL	Strategic Why Mimic picks; execution_mode: why_mimic; Slide Intelligence on mimic_v1
FLOW_TOP_PERFORMER_MIMIC_VIDEO	Video	top_performer_video	Planning alias only — routes to HeyGen; no pixel mimic render

Routing rules:

- Image mimic **fails early** if more than one reference frame resolves.

- Carousel mimic handles multi-slide archives, video-slide skipping, and promotional-slide filtering.
- **Video top performers** do not use pixel mimic — Nemotron
`top_performer_video` insights route to **HeyGen** via
`resolveTopPerformerVideoHeygenRoute()` (`src/domain/top-performer-video-heygen-routing.ts`). Creation packs include
`top_performer_video_knowledge` when jobs ground to video insights.
- **Visual-first carousel ideas** (`carousel_style: visual_first` or `mixed`)
plan `FLOW_VISUAL_FIRST_CAROUSEL` when enabled. Prep is `new-visual-carousel-prep.ts` — **no** top-performer `reference_items` ; always
`execution_mode: new_visual` , `mode: carousel_visual` , T2I via
`analysis_t2i` . BVS defaults on. **Review:** TP-grounded workbench (layer editor, slide regen) — **no** original-vs-generated compare.
- **Why Mimic carousel** plans `FLOW_WHY_MIMIC_CAROUSEL` with SIL-driven
copy/prompts (`execution_mode: why_mimic`). Same render engine; requires
substantive SIL when `WHY_MIMIC_REQUIRE_SUBSTANTIVE_SIL` is enabled.
- **Manual mimic carousel picks** plan `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL`
(`execution_mode: classic`). **Review:** same workbench **plus** original-vs-
generated compare.
- Mimic flows use **separate planning lanes** (`mimic_image` , `mimic_carousel` ,
`visual_first_carousel`) so they can run **in parallel** with standard
`FLOW_CAROUSEL` without sharing the `max_carousel_jobs_per_run` cap.

Code: `src/domain/top-performer-mimic-flow-types.ts` ,
`src/decision_engine/format-routing.ts`

3. Prerequisites (data + configuration)

3.1 Upstream data

1. **Top-performer ingest** — Creative Intelligence pipeline archives inspection media to Supabase (`stored_inspection_media_json` on visual-guidelines entries). See `docs/CREATIVE_INTELLIGENCE.md` .
2. **Signal pack** with `derived_globals_json.visual_guidelines_pack_v1` — per-insight rows carrying aesthetic analysis, slide transcripts, `mimic_evaluation` , and archived media paths.
3. **Ideas grounded** to top-performer `insights_id` values
(`ideas_json[].grounding_insight_ids`). Each insight id may ground **at most one** idea.

4. For image mimic expansion at planning time: either explicit `target_flow_type` , `manual_mimic_pick` with `mimic_kind: "image"` , or eligibility via `mimicImageReferenceEligible()` (single-frame deep reference).

3.2 Environment flags

Variable	Default	Purpose
MIMIC_IMAGE_ENABLED	false	Master switch — mimic draft/render paths stay off when false
MIMIC_IMAGE_PROVIDER	bfl	bfl dashscope nvidia openai
MIMIC_IMAGE_BFL_MODEL	flux-2-klein-4b	BFL model slug (flux-2-flex for typography-tuned)
MIMIC_VISUAL_SIMILARITY_PCT	70	How closely reference_edit should follow pixels ($\leq 25\%$ → low fidelity on OpenAI)
MIMIC_IMAGE_INPUT_MODE	reference_edit	reference_edit (image-to-image) or analysis_t2i (text-to-image from Flux prompts)
MIMIC_IMAGE_DEFAULT_SIZE	1024x1536	Instagram portrait default
OPENAI_API_KEY	—	Required for copy generation (all mimic flows)
Provider API keys	—	BFL_API_KEY, DASHSCOPE_API_KEY, NVIDIA_NIM_API_KEY, or OPENAI_API_KEY for render

3.3 Project settings

- Enable `FLOW_TOP_PERFORMER_MIMIC_IMAGE` , `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` , and/or `FLOW_VISUAL_FIRST_CAROUSEL` in allowed flow types.
- Set plan cap > **0** for the mimic flow (flows are seeded **disabled** via `seedMimicFlowTypesSkeleton()`).
- Optional per-project overrides in `project_system_constraints` : `mimic_image_bfl_model` , `mimic_visual_similarity_pct` , `mimic_carousel_text_via_flux` , `mimic_image_input_mode` .

Migrations: `060_mimic_flow_types.sql` , `061_mimic_flow_prompts.sql` , `062 – 069` (copy rules, grounding, BFL model, render settings).

4. Core payload contracts

4.1 `generation_payload.mimic_v1` (render source of truth)

Written during **mimic prep** (before or after LLM, depending on step). Consumed at **render**.

Field	Meaning
execution_mode	classic why_mimic new_visual — lane semantics (see flow table §2)
mode	image_full template_bg carousel_visual
mode_override	Reviewer override from signal pack (mimic_mode_overrides)
source_insights_id	Top-performer insight (new_visual uses "new_visual")
reference_items[]	Archived frames (empty for new_visual)
archive_reference_items[]	Full deck before promo/video filtering
slide_plans[]	Per output slide: render_mode (hbs full_bleed), reference_index
visual_guideline	Slim vision row (no signed URLs)
twist_brief	{ visual_only: true, legal_note }
docai_layer_positions	Reviewer layout edits (preserved across re-renders)
flux_image_prompts	Per-slide T2I prompts when image_input_mode = analysis_t2i
slide_intelligence	Why Mimic SIL bundle (read-only)
bvs_enabled, bvs_bible_snapshot, bvs_render_plan	Brand Visual System snapshot + invented-plate plan for template_bg

Code: src/domain/mimic-payload.ts

4.2 mimic_carousel_package (review / operator snapshot)

All **TP-grounded carousel render flows** (isTpGroundedCarouselRenderFlow()):
FLOW_TOP_PERFORMER_MIMIC_CAROUSEL , FLOW_VISUAL_FIRST_CAROUSEL ,
FLOW_WHY_MIMIC_CAROUSEL . Stored in draft_package_snapshot :

Slice	Content
copy	LLM slides, caption, hashtags
render_plan	strategy: template_background Or per_slide_mimic
visual_reference	Storage paths, reference_items
visual_guideline	Format pattern, deck system, mimic_evaluation, per-slide cues

twist_brief	Legal / visual-only note
-------------	--------------------------

`mimic_v1` remains authoritative for render; the package is for Review UI, content log, and operators.

Code: `src/domain/mimic-carousel-package.ts` , `src/services/draft-package-contract.ts`

4.3 Supporting payload keys

Key	When set	Purpose
mimic_render_context	Before LLM	Slide count targets, copy_before_visual_mimic, template path hints
mimic_job_grounding	Before LLM	slide_copy_layout — per-slide reference text, typography, copy slots
template_storage_decision	Before LLM	Whether backgrounds go to project template library
template_backgrounds_prepared_at	Pre-copy (template_bg)	Background plates extracted before copy gen
candidate_data	Planning	grounding_insight_ids, idea_id, use_brand_visual_system for lineage / BVS

4.4 Brand Visual System (`generation_payload.bvs_v1`)

Versioned `brand_bibles` (`brand_bible_v1`) per project. When `candidate_data.use_brand_visual_system === true` (visual-first defaults on), plan time stamps `bvs_v1` onto `generation_payload` and enriches `mimic_v1` with `bvs_enabled` , `bvs_bible_snapshot` , `bvs_render_plan` .

Piece	Role
brand_bibles table	Versioned bible JSON per project (migrations/078_brand_bibles.sql)
bvs_v1 on job	Frozen snapshot: enabled, bible_version, bible_snapshot
bvs_render_plan ON mimic_v1	Invented plates for template_bg when BVS + listicle (vs reference strip)
bvs-render-overlays.ts	Logo/frame overlays + palette injection at carousel render

Code: `src/domain/brand-bible.ts` , `bvs-v1.ts` , `bvs-render-plan.ts` , `src/repositories/brand-bibles.ts`

5. Mimic modes and render strategies

5.1 Mode classification

`classifyMimicMode()` (`src/services/mimic-mode-classifier.ts`) picks the render path for **classic** and **why_mimic** lanes. **new_visual** jobs bypass TP reference resolve and are forced to `mode: carousel_visual` + `execution_mode: new_visual` in `new-visual-carousel-prep.ts` .

Priority (classic / why_mimic):

1. Manual `mode_override` (reviewer on signal pack)
2. Nemotron `mimic_evaluation.recommended_mode` :
 - `text_on_template` → `template_bg`
 - `full_bleed_visual` → `carousel_visual`
3. Heuristic fallback (text density, `format_pattern` , unified background cues)

Mode	Typical reference	Render approach
<code>image_full</code>	1 frame	Single <code>STATIC_IMAGE</code> via image edit; on-image copy from LLM
<code>template_bg</code>	Text-heavy listicle / educational	Extract background plates → HBS (<code>carousel_mimic_bg.hbs</code>) or Sharp composite + text overlay
<code>carousel_visual</code>	Image-led / mixed deck	Per-slide art-only mimic (~70% similarity default) + HBS text overlay where <code>on_screen_text_transcript</code> exists

Slide render modes:

- `hbs` — Puppeteer Handlebars template overlays LLM copy; typography from Nemotron `text_blocks` + Document AI merge.
- `full_bleed` — Art-only image model output (no baked-in text); copy overlaid in a later HBS/DocAI step if needed.

Slides with **any** on-screen text transcript use **hbs**, not image-model typography.

5.2 Render plan strategies (carousel package)

<code>render_plan.strategy</code>	<code>mimic_v1.mode</code>	Behavior
<code>template_background</code>	<code>template_bg</code>	Reuse/extract bg plate → overlay text
<code>per_slide_mimic</code>	<code>carousel_visual</code>	Per-slide visual plate, then text overlay

6. End-to-end pipeline (operator workflow)

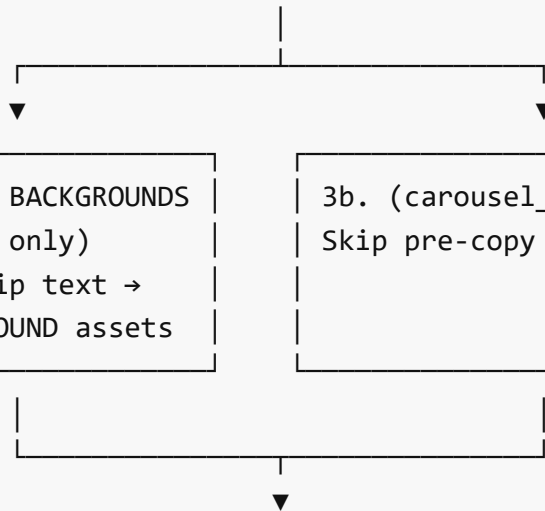
1. PLAN RUN

Mimic candidates compete in parallel with FLOW_CAROUSEL when enabled



2. RESOLVE REFERENCE (before LLM)

resolveMimicReferenceFromLineage → mimic_v1 + mimic_render_context
+ template_storage_decision



4. GENERATE JOBS (OpenAI copy)

- template_bg: full slide copy
- full_bleed: caption/hashtags + short hooks
- mimic_carousel_package snapshot

Status → GENERATED



5. REVIEW (copy + mimic metadata – often no final assets yet)

Review app: MimicCarouselEdits, mode overrides, DocAI layer editor



6. RENDER

- `template_bg`: bg plate + HBS/DocAI text layers
- `carousel_visual`: per-slide plates + overlays
- `image_full`: single image edit with on-image copy

Status → `IN_REVIEW` (pending human approval)

6.1 Key functions by stage

Stage	Primary modules
Reference resolve	<code>mimic-reference-resolver.ts</code> , <code>mimic-reference-eligibility.ts</code>
Pre-copy prep	<code>ensureMimicReferenceBeforeCopyGeneration</code> , <code>ensureMimicTemplateBackgroundsBeforeCopy</code> in <code>mimic-draft-prep.ts</code>
LLM copy	<code>llm-generator.ts</code> , <code>carousel-mimic-copy-policy.ts</code> , <code>mimic-copy-coherence.ts</code>
Post-copy snapshot	<code>prepareMimicDraftPackage</code> in <code>mimic-draft-prep.ts</code>
Carousel render	<code>mimic-carousel-render.ts</code> , <code>mimic-template-bg-render.ts</code> , <code>mimic-docai-overlay-layout.ts</code>
Image render	<code>mimic-image-job.ts</code> , <code>mimic-image-provider.ts</code>
Orchestration	<code>job-pipeline.ts</code>

7. Reference resolution (deep dive)

7.1 Lineage

Every mimic job links to a **signal pack** via `generation_payload` lineage (`getJobLineageByTaskId`). Resolution uses:

1. `candidate_data.grounding_insight_ids` (preferred), or
2. Job grounding rows from the signal pack.

7.2 Analysis tiers

Flow	Expected tier	Fallback
Image mimic	<code>top_performer_deep</code>	<code>top_performer_carousel</code> (flagged <code>reference_tier_fallback</code>)
Carousel mimic	<code>top_performer_carousel</code>	none

Image mimic **skips** entries with **>1** reference frame.

7.3 Reference items

Archived frames come from `stored_inspection_media_json` / `inspection_media` on the visual-guidelines entry. Each item carries:

- `vision_fetch_url` — fetch at render (re-signed from `bucket` + `object_path` when URLs expire)
- `source_slide_index` — 1-based position in original Instagram deck
- `is_video_slide` — skipped at render

`normalizeMimicReferenceItems()` enforces 1-based contiguous indexes for render plans.

7.4 Promotional / video slide handling

- **Video slides** — omitted from mimic reference set; indices tracked in `video_slide_indices`.
- **Promotional slides** — `filterPromotionalSlidesFromMimicPayload()` drops slides tagged `self_promo`, `product_pitch`, or high `brand_specificity`.
- **Skip indices** — `mimic_evaluation.skip_slide_indices` and `content_slide_indices` drive which slides count toward copy/render.

8. LLM copy generation (deep dive)

8.1 Prompt templates

Migration `061_mimic_flow_prompts.sql` seeds:

- `MIMIC__Top_Performer_Carousel_v1` → `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL`
- `MIMIC__Top_Performer_Image_v1` → `FLOW_TOP_PERFORMER_MIMIC_IMAGE`

Both **inherit the `FLOW_CAROUSEL` output schema** so QC and validation stay unchanged.

8.2 Signal pack budgeting (mimic-only)

Mimic prompts are large (vision grounding + top-performer knowledge). **Do not pass full `ideas_json`** (30+ rows). Required behavior:

1. Filter `ideas_json` to the job's planned idea only.
2. Match `idea_id` with underscore boundaries (avoid `idea_12` matching `idea_1`).
3. Slim `visual_guidelines_pack_v1` — strip `inspection_media` / signed URLs from LLM context.
4. Cap hashtag leaderboard size.

Code: `llm-creation-pack-budget.ts` , `llm-generator-helpers.ts`
(`mimicFlowOnly: true`)

8.3 Copy branches

Mode	Copy expectation
<code>template_bg</code>	Full per-slide headline/body matching reference structure
<code>carousel_visual / full_bleed</code>	Caption, hashtags, short hooks; reference text length scaled by <code>MIMIC_FULL_BLEED_COPY_REFERENCE_SCALE</code>
<code>image_full</code>	Caption, hook, hashtags; on-image copy for render

8.4 Grounding blocks

`mimic_job_grounding.slide_copy_layout` provides per-slide:

- `reference_on_screen_text` (structure/length guide only — output must be fresh)
- `visual_description` , `layout_template` , `text_blocks` , `copy_slots_v1`

`assertMimicCopyDiffersFromReference()` guards against verbatim copying of reference wording.

Optional: `MIMIC_COPY_COHERENCE_LLM` runs a coherence pass after generation.

8.5 Context guards

Mimic carousel prompts have char budgets (`LLM_MIMIC_*_MAX_CHARS` , `MIMIC_CAROUSEL_SYSTEM_PROMPT_MAX_CHARS`) with emergency shrinking in `shrinkMimicCarouselPromptsIfNeeded()` .

9. Visual render (deep dive)

9.1 Image providers

`mimic-image-provider.ts` abstracts:

Provider	Typical model	Notes
bfl	flux-2-klein-4b, flux-2-flex	Default; async poll; flex supports steps/guidance
dashscope	qwen-image-edit-max	Alibaba; size 1024*1536
nvidia	qwen/qwen-image-edit	NIM OpenAI-compatible /images/edits
openai	gpt-image-1	input_fidelity, quality

All prompts pass through `finalizeMimicImageModelPrompt()` with **art-only guard** (no on-image text unless explicitly allowed).

9.2 Template background path (`template_bg`)

1. **Pre-copy** (optional): `ensureMimicTemplateBackgroundsBeforeCopy` extracts cover/body/CTA plates → assets type `MIMIC_BACKGROUND` .
2. **Reusable templates**: when `mimic_evaluation.template_storage_quality = reusable` , plates also go to project template library (`mimic-template-library.ts`).
3. **Render**: Puppeteer renders `carousel_mimic_bg.hbs` with LLM copy + Nemotron typography, **or** Sharp composite (`carousel-composite-render.ts`) when `CAROUSEL_COMPOSITE_ENABLED=true` .

9.3 Carousel visual path (`carousel_visual`)

1. Per slide: `generateMimicSlideImage()` with art-only prompt (`mimic-prompt-builder.ts`).
2. Reference frame selected via `referenceItemForMimicSlide()` using `source_slide_index` alignment.

3. Output assets: `MIMIC_VISUAL_PLATE` per slide.
4. Text overlay: Document AI + Nemotron merge → HBS or DocAI fit page (`mimic-docai-fit-page.ts` , `mimic-docai-text-contrast.ts`).

9.4 Image full path (`image_full`)

`processImageMimicJob()` :

1. Asserts single reference frame.
2. Reads on-image copy from `generated_output` (`onImageCopyForMimicRender`).
3. One image edit → `STATIC_IMAGE` asset.
4. Job status → `IN_REVIEW` .

9.5 Re-render and text-only reprint

Reviewers can adjust `docai_layer_positions` and trigger **text-overlay reprint** (`MIMIC_TEXT_OVERLAY_REPRINT_PHASE`) without re-running image models — requires existing `MIMIC_BACKGROUND` or `MIMIC_VISUAL_PLATE` assets.

9.6 `analysis_t2i` mode

When `MIMIC_IMAGE_INPUT_MODE=analysis_t2i` :

- Per-slide Flux prompts generated at copy time (`mimic-flux-image-prompts.ts` , optional `MIMIC_FLUX_PROMPT_LLM`).
 - Render uses text-to-image (no reference pixels) — useful when reference edit is undesirable.
-

10. Planning and candidate materialization

10.1 Image mimic expansion guard

`shouldExpandTopPerformerMimicImageForRow()` returns true when:

- `manual_mimic_pick === true` and `mimic_kind === "image"` , or
- `target_flow_type === FLOW_TOP_PERFORMER_MIMIC_IMAGE` , or
- Grounding resolves to single-frame deep reference (`mimicImageReferenceEligible`).

Otherwise image mimic candidates are **skipped** at expansion.

10.2 Parallel lanes

`format-routing.ts` assigns:

- `mimic_carousel` lane for carousel mimic
- `mimic_image` lane for image mimic
- Standard `carousel` lane for `FLOW_CAROUSEL`

This allows one idea to produce both a templated carousel job and a mimic carousel job when both flows are enabled.

10.3 Creative intelligence boost

Ideas with `grounding_insight_ids` starting with `ci_` get higher `past_performance` score
(`CREATIVE_INTEL_PLANNER_PAST_PERFORMANCE_BOOST`).

11. Review app integration (`apps/review`)

Feature	API / component
Mimic carousel edits	<code>MimicCarouselEdits.tsx</code>
DocAI layer position editor	<code>MimicDocAiLayerPositionEditor.tsx</code>
Mode override per insight	<code>POST .../signal-packs/:packId/mimic-mode-override</code>
Mimic image audits	<code>GET .../mimic-image-audits</code>
Layer editor panel	<code>MimicCarouselLayerEditorPanel.tsx</code>

Review app is a **client** of Core APIs — `content_jobs` in Postgres remains source of truth.

12. Asset types

Asset type	When created
<code>MIMIC_BACKGROUND</code>	Template-bg plate (cover/body/CTA)
<code>MIMIC_VISUAL_PLATE</code>	Per-slide art-only plate (carousel_visual)
<code>STATIC_IMAGE</code>	Single-frame image mimic output
<code>CAROUSEL_IMAGE</code>	Final composed slide images

13. Configuration reference (extended)

Variable	Purpose
MIMIC_CAROUSEL_TEXT_VIA_FLUX	Legacy Flux text-bake (pipeline forces DocAI/HBS overlay today)
MIMIC_USE_PROJECT_BRAND_PALETTE	Inject brand colors into mimic prompts
MIMIC_USE_BRAND_IMAGE_STYLE_HINTS	Brand image style in prompts
MIMIC_FLUX_PROMPT_LLM	OpenAI authors Flux T2I prompts
MIMIC_FULL_BLEED_COPY_REFERENCE_SCALE	Scales reference text length targets
MIMIC_COPY_CHAR_SLACK	± chars tolerance on copy length guards
MIMIC_IMAGE_BFL_FALLBACK_DASHSCOPE	BFL failure → DashScope
MIMIC_IMAGE_NVIDIA_FALLBACK_OPENAI	NVIDIA failure → OpenAI
CAROUSEL_COMPOSITE_ENABLED	Sharp composite vs Puppeteer-only

14. Key file map

Area	Files
Flow type constants	src/domain/top-performer-mimic-flow-types.ts
Payload types	src/domain/mimic-payload.ts, src/domain/mimic-carousel-package.ts
Mode classification	src/services/mimic-mode-classifier.ts, src/domain/mimic-text-heavy.ts
Reference resolve	src/services/mimic-reference-resolver.ts
Draft prep	src/services/mimic-draft-prep.ts
LLM copy	src/services/llm-generator.ts, src/services/carousel-mimic-copy-policy.ts
Image generation	src/services/mimic-image-provider.ts, src/services/mimic-image-job.ts
Carousel render	src/services/mimic-carousel-render.ts
Prompts	src/services/mimic-prompt-builder.ts
Project config	src/services/mimic-project-config.ts
Pipeline orchestration	src/services/job-pipeline.ts
HBS template	services/renderer/templates/carousel_mimic_bg.hbs
Docs	docs/MIMIC_IMAGE_FLOWS.md, docs/CREATIVE_INTELLIGENCE.md

15. Invariants and pitfalls

1. `task_id` is the execution key across `content_jobs` , `assets` , `job_drafts` , reviews.
 2. `mimic_v1` is render truth; `mimic_carousel_package` is operator/review snapshot — do not conflate with `carousel_package` .
 3. **Never** pass full `ideas_json` into mimic LLM prompts.
 4. **Image mimic** requires exactly one reference frame — multi-frame references must use carousel mimic.
 5. `MIMIC_IMAGE_ENABLED=0` blocks render even if flow types are enabled on the project.
 6. **Copy generation** still requires `OPENAI_API_KEY` regardless of image provider.
 7. **Re-sign URLs** at render — `refreshMimicPayloadReferenceUrls()` handles expired Supabase signed URLs.
 8. **Video slides** in archived carousels are never mimicked.
 9. **Mode overrides** on signal packs take precedence over automatic classification.
 10. **Do not double-submit** HeyGen or other provider renders — mimic uses its own render path via `job-pipeline.ts` .
-

16. Glossary

Term	Definition
Top performer	High-performing archived social post used as visual/copy reference
Visual guidelines pack	<code>visual_guidelines_pack_v1</code> on signal pack derived globals
Nemotron	NVIDIA vision model producing <code>aesthetic_analysis_json</code> , <code>text_blocks</code> , <code>mimic_evaluation</code>
Art-only mimic	Image model prompt forbids baking text into pixels
Template bg	Text-heavy deck: extract clean background, overlay fresh copy
Full bleed	Image-led slide: mimic visual plate, overlay text separately
Grounding	Link from signal-pack idea to <code>insights_id</code> of top-performer analysis
Twist brief	Legal constraint: visual pattern only, no verbatim logos/faces

17. Related HTTP endpoints (selection)

- Creative intelligence ingest: `POST /v1/creative-intelligence/:slug/top-performers/ingest`

- Signal pack styling merge: `POST /v1/creative-intelligence/:slug/signal-packs/:id/styling`
- Mimic mode override: signal pack routes in `src/routes/signal-packs.ts`
- Mimic text overlay lab: `src/routes/mimic-text-overlay-lab-routes.ts`
- Review contract paths: covered by `src/routes/review-contract.test.ts`

Cross-linked from `README.md` , `AGENTS.md` , and `docs/CAF_CORE_COMPLETE_GUIDE.md` . For repo-wide doc index see `docs/Layers/README.md` .

Included: docs/MIMIC_IMAGE_FLOWS.md

Top-performer mimic flows (quick reference)

Optional pipeline lanes for carousel/image mimic, **new visual** carousels, and **Why Mimic**. Full detail: [MIMIC FLOWS COMPLETE GUIDE.md](#). Repo truth: [CAF CURRENT STATE CONTEXT PACK.md](#) §11.

Updated 2026-07: Visual-first (`FLOW_VISUAL_FIRST_CAROUSEL`) is **not** TP frame replication — it uses `execution_mode: new_visual` with idea + BVS and empty `reference_items` .

Flow types

Flow	Format	Lane
<code>FLOW_TOP_PERFORMER_MIMIC_IMAGE</code>	Single post	top_performer_deep (exactly one frame)
<code>FLOW_TOP_PERFORMER_MIMIC_CAROUSEL</code>	Carousel	Manual mimic picks; <code>execution_mode: classic</code>
<code>FLOW_VISUAL_FIRST_CAROUSEL</code>	Carousel	New visual — idea + BVS; <code>execution_mode: new_visual</code> ; no TP frames
<code>FLOW_WHY_MIMIC_CAROUSEL</code>	Carousel	SIL-driven; <code>execution_mode: why_mimic</code>
<code>FLOW_TOP_PERFORMER_MIMIC_VIDEO</code>	Video	Routes to HeyGen (<code>FLOW_VID_*</code>) — not pixel mimic

TP-grounded carousel lanes share render/copy (`isTpGroundedCarouselRenderFlow()`) but have **separate** plan caps and prompts.

Enable (both required)

- 1. **Env:** `MIMIC_IMAGE_ENABLED=1` , `OPENAI_API_KEY` (copy), and a render provider key for `MIMIC_IMAGE_PROVIDER` (default `bfl` → `BFL_API_KEY` ; also `dashscope` , `nvidia` , `openai`).
- 2. **Project:** enable mimic flow types + plan cap > 0 (seeded **disabled** by default).

Payload keys

Key	Role
mimic_v1	Render source of truth — execution_mode, mode, references, slide plans, BVS slices
bvs_v1	Frozen Brand Visual System snapshot when enabled
mimic_carousel_package	Review snapshot for all TP-grounded carousel render flows — not carousel_package
mimic_render_context	Copy-time hints (slide count, template path)
mimic_job_grounding	Per-slide layout for copy LLM

mimic_v1.execution_mode

Value	Flow	Notes
classic	FLOW_TOP_PERFORMER_MIMIC_CAROUSEL	TP reference frames
new_visual	FLOW_VISUAL_FIRST_CAROUSEL	No reference_items; prep in new-visual-carousel-prep.ts
why_mimic	FLOW WHY_MIMIC_CAROUSEL	SIL on slide_intelligence

Mimic modes (mimic_v1.mode)

Mode	When	Render
image_full	Single reference frame	One STATIC_IMAGE via image edit + LLM on-image copy
template_bg	Text-heavy / listicle deck	Background plate (reference strip or BVS-invented T2I) → HBS/DocAI overlay
carousel_visual	Image-led carousel	Per-slide art-only plates + HBS/DocAI overlay

New visual is always `carousel_visual` + T2I. Classifier (`classifyMimicMode()`) applies to classic/why mimic only.

Text invariant: image models produce **art-only plates**. Copy is composited via **HTML/CSS** (Puppeteer HBS or DocAI `docai_layer_positions`).

`MIMIC_CAROUSEL_TEXT_VIA_FLUX` is **ignored** at render.

Brand Visual System (BVS)

- `brand_bibles` table — versioned `brand_bible_v1` per project.
- Snapshoted to `generation_payload.bvs_v1` at plan when `use_brand_visual_system` is true.
- Visual-first defaults BVS on. `mimic_v1.bvs_render_plan` can invent `template_bg` plates.

Operator workflow

1. **Plan run** — mimic flows compete with `FLOW_CAROUSEL` when both enabled.
2. **Prep** — classic/why: `mimic-draft-prep.ts` ; new visual: `new-visual-carousel-prep.ts` .
3. **Generate** — OpenAI copy; TP-grounded carousel → `mimic_carousel_package` snapshot.
4. **Review** — layer editor, live preview, brand styling panel; manual mimic has compare row; visual-first does not.
5. **Reprint** (cheap) vs **regenerate slide** (expensive) for wrong visuals.
6. **Render** — image provider + Puppeteer/DocAI overlays → **IN_REVIEW**.

Prerequisites

- **Classic / Why Mimic:** top-performer archive in Supabase; signal pack `visual_guidelines_pack_v1` + grounded ideas.
- **New visual:** idea + BVS; no TP archive required for reference frames.
- See [CREATIVE INTELLIGENCE.md](#) for ingest.

Config (defaults from `src/config.ts`)

Variable	Default
<code>MIMIC_IMAGE_ENABLED</code>	<code>false</code>
<code>MIMIC_IMAGE_PROVIDER</code>	<code>bfl</code>
<code>MIMIC_VISUAL_SIMILARITY_PCT</code>	<code>70</code>
<code>MIMIC_IMAGE_INPUT_MODE</code>	<code>reference_edit</code> (<code>analysis_t2i</code> for new visual / invented plates)

WHY_MIMIC_REQUIRE_SUBSTANTIVE_SIL	true
MIMIC_LAYOUT_QA_ENABLED	true

Per-project overrides: `project_system_constraints.mimic_*` (migrations `066` – `069`).

Key modules

Area	Path
Classic prep	src/services/mimic-draft-prep.ts
New visual prep	src/services/new-visual-carousel-prep.ts
BVS	src/domain/brand-bible.ts, bvs-v1.ts, bvs-render-plan.ts
Carousel render	src/services/mimic-carousel-render.ts, bvs-render-overlays.ts
Payload types	src/domain/mimic-payload.ts, mimic-carousel-package.ts

Migrations

`060` – `078` — flow types, prompts, BVS (`078_brand_bibles`), Why Mimic (`074`), visual-first (`070`), job outcomes (`075`).

Included:

`docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md`

Mimic Text Placement — Automation Path (Design)

Status: Phase A implemented (2026-06). Post-composite analyzer, auto-reprint loop (cap N), Review gate (`BLOCKED` when layout fails), regen state on `render_state.carousel_regenerate` , per-slide badges in Review. Vision QA and human suggestion pre-load remain future work.

For AI assistants: Cursor rule [.cursor/rules/mimic-text-placement-automation.mdc](#) — read when the user asks to automate mimic text placement.

The real problem (not “learn placement once”)

Mimic carousel text placement has **two different inputs** that must not be conflated:

Input	Stable?	What it tells you
Reference top-performer slide (OCR)	Yes, per mimic job	Where text <i>was</i> on the <i>original</i> — roles, rough geometry, copy structure
Generated art plate (Flux/Qwen)	No — changes every regen	Where faces, products, logos, and busy texture land <i>on this slide</i>

Reference OCR → initial boxes is only a **seed**. It cannot be trusted as final placement because the model may put the subject exactly where the seed box sits. That is why a human opens the layout editor today: **on this specific rendered slide**, text is covering something important or sitting on illegible background.

Automation cannot primarily mean “learn from past human moves.” Past jobs had different plates; a delta from slide A does not predict slide B. Automation must mean:

```
seed boxes (reference OCR) → HTML reprint → final composite PNG
  → analyze THIS composite
  → “box X is bad here; move to Y”
  → reprint (cheap) → re-analyze or route to human
```

The human editor remains the **exception handler** when the post-render loop cannot reach confidence. It is not a training-data farm for a static placer.

What exists today (and what it is *not*)

Seed path (before human):

1. Document AI OCR on the **reference** slide.
2. [src/services/mimic-docai-overlay-layout.ts](#) maps copy → `docai_text_layers` (roles, single vs multi-line).
3. [services/renderer/mimic-docai-fit.js](#) fits font size and resolves **box–box** collisions at render time.
4. [services/renderer/mimic-docai-contrast.js](#) samples the **background plate** (not the composite) for text color/contrast.

Human path:

- [MimicDocAiLayerPositionEditor.tsx](#) writes corrections to `docai_layer_positions`.

Cheap iteration:

- **Reprint** recomposites HTML on stored plates (no Flux). This is the loop motor for any automated fixer.

Gap: Nothing today re-examines the **printed slide** (image + overlay together) and asks “did we ruin this slide?” Pre-render `mimic_avoid_center_subject` uses heuristics on the plate only — it cannot see that headline landed on a face after this particular generation.

The automation spine: post-composite placement QA

This is the step automation must solve. Everything else is supporting.

```
flowchart TD
    REF[Reference OCR + solver seed] --> REP1[Reprint composite]
    PLATE[Generated plate] --> REP1
    REP1 --> PNG[Final slide PNG]
    PNG --> QA[Post-composite analyzer]
    QA -->|pass| DONE[Auto-approve or skip human]
    QA -->|fixes| PATCH[Patch docai_layer_positions]
    PATCH --> REP2[Reprint]
    REP2 --> PNG
    QA -->|low confidence| HUMAN[Human layout editor]
    HUMAN --> PATCH
```

What the analyzer looks at

Re-running “OCR” on a slide that already contains your text is noisy (it reads overlay + background). The useful signal is **placement QA on the composite**, not re-extracting copy:

Check	Question	Typical failure
Saliency / subject overlap	Do text boxes intersect faces, products, logos, or high-detail regions?	Headline on a face
Legibility	Is contrast sufficient in the box region (extend existing contrast pass to composite)?	Light text on light sky
Box collision	Do boxes overlap each other after fit?	Already partly in <code>mimic-docai-fit.js</code>
Margin / safe zone	Are boxes clipped or hugging edges?	CTA cut off
Copy completeness	Is all intended copy visible (not hidden/off-canvas)?	Solver dropped a line

Outputs are still `docai_layer_positions` **patches** — same schema as the human editor:

```
{
  "layer_key": "headline@0",
```

```

"x_px": 96, "y_px": 280, "w_px": 888, "h_px": 200,
"source": "post_render_qa", // ocr | post_render_qa | human |
vision
"fix_reason": "subject_overlap" // optional: why this move was
proposed
}

```

Provenance values:

- **ocr** — seed from reference layout solver (default, pre-QA).
- **post_render_qa** — moved by the composite analyzer (automation).
- **human** — reviewer edit in the drag editor.
- **vision** — optional vision-LLM suggestion before first print (see below).

Analyzer implementation options (ranked)

1. Deterministic composite QA (build first)

Render once, run vision/heuristics on the PNG:

- Subject/saliency map on the **composite** (or plate + known box mask).
- Penalize boxes whose area overlaps salient pixels above a threshold.
- Search nearby positions (grid or slide “quiet zones”) that preserve copy and clear overlap.
- Reuse contrast sampling from `mimic-docai-contrast.js` against the region under each box.

No training data required; works per slide, per generation.

2. Vision-model QA (suggestion / hard cases)

Send the **composite PNG** + box list + target copy to a vision model: “which boxes should move and where?” Return the same JSON schema with `source: "post_render_qa"`. Use when deterministic QA is uncertain.

3. Vision-model pre-placement (optional, not sufficient alone)

Propose initial boxes from **generated plate + copy** before first print (`source: "vision"`). Still must pass through step 1 after print — unpredictable generation means pre-placement alone is never enough.

4. Stronger reference seed (supporting only)

Improve `mimic-docai-overlay-layout.ts` (copy anchoring, roles, center-avoid). Reduces how often QA must move boxes; does **not** replace post-composite QA.

5. Learning from human corrections (deprioritized)

Storing OCR seed vs human final across jobs is optional telemetry for improving heuristics or few-shot prompts. It is **not** the primary automation path because plates differ every time.

Confidence gate (when to skip the human)

After each reprint, compute a **placement score** from composite QA:

- overlap with salient regions (weight high),
- contrast legibility,
- inter-box collision,
- margin violations.

Score	Action
High	Auto-approve slide for review queue (or auto-publish path later)
Medium	Apply QA patches, reprint once more, re-score
Low	Open human layout editor with QA suggestions pre-loaded

Implemented policy (2026-06): auto-reprint runs on soft findings (collision, margin, subject zone).

All jobs reach **IN_REVIEW** ; **layout_qc.review_attention** + thumb badges flag slides that need layout editor polish before publish. Layout QA does **not BLOCK** jobs by default (**MIMIC_LAYOUT_QA_BLOCK_*** both default off).

This turns the editor from “default for every slide” into “only when this generation fought us.”

Design principle: one schema, two phases

Keep a single box schema for seed, QA fixes, and human edits so the pipeline stays one path:

Phase	Who writes boxes	When
Seed	Reference OCR + solver	After plate exists, before or after first print
QA fix	Post-composite analyzer	After each print — the automation step
Human override	Reviewer	When QA confidence is low or brand judgment needed

The editor overhaul already emits this shape; **source** (and optional **fix_reason**) records which phase last moved a box. Persisting **source** through Core is small future plumbing.

How current product pieces support this

Piece	Role in post-render loop
HTML overlay + reprint	Cheap iterate: QA → patch → reprint without Flux
<code>doci_layer_positions</code>	QA and human write the same patch format

Brand palette swatches (editor 1.5)	QA can auto-pick legible color_hex from palette
Regenerate route (similarity / reference)	Only when the plate is wrong, not placement
mimic-docai-contrast.js	Extend to sample composite under each box
Human layout editor	Exception UI; show QA moves as suggestions to accept/nudge

Suggested build order

1. **Composite overlap detector** — given PNG + box list, flag boxes intersecting salient/busy regions (deterministic or light vision model).
2. **Box nudge solver** — search local positions that clear flags; emit `post_render_qa` patches.
3. **Reprint loop in pipeline** — after mimic render, run QA → patch → reprint (cap N iterations).
4. **Confidence gate** — score → auto-pass vs human queue; surface QA suggestions in editor.
5. *(Optional)* Vision QA for hard slides; *(optional)* telemetry from human finals.

Not in scope yet

- Vision-model QA for hard slides (deterministic analyzer is live).
- Auto reprint loop without human-in-the-loop approval on low-confidence slides (auto-reprint runs in pipeline; human editor still exception path).
- Training / finetuning on historical correction pairs.

Included: docs/CREATIVE_INTELLIGENCE.md

Creative intelligence (top performers)

Project-agnostic layer: ingest reference media from high-performing social posts, store assets (Supabase when configured), run multimodal vision analysis, persist structured insights, expose styling cues on signal packs, and optionally mint carousel `.hbs` templates for the Puppeteer renderer.

Data flow

1. `POST /v1/creative-intelligence/:project_slug/top-performers/ingest` — download/store slides or video (+ ffmpeg frames when

- available), optional inline OpenAI vision → `creative_visual_analyses` + `creative_insights` with stable `insight_ref` values like `ci_<hex>` .
2. `POST /v1/creative-intelligence/:project_slug/insights/generate` — aggregate multiple completed analyses into additional `creative_insights` (`ci_agg_*`).
 3. `POST /v1/creative-intelligence/:project_slug/signal-packs/:signal_pack_id/styling` — merges `creative_design_intelligence_v1` and `top_performer_styling_cues_v1` into `signal_packs.derived_globals.json` .
 4. `POST .../mimic-carousel-template` — writes `services/renderer/templates/<name>.hbs` (from `carousel_notes_app_minimal.hbs` + palette) and pins the template on the project.
 5. `POST .../jobs/:task_id/apply-template` — sets `generation_payload` render keys for that job.

Tables (`migrations/055_creative_intelligence.sql`)

Table	Purpose
<code>caf_core.creative_source_assets</code>	Media rows (slides, video blob, thumbnail, <code>extracted_frame</code>), <code>source_group_id</code> , <code>metrics/metadata</code> JSON.
<code>caf_core.creative_visual_analyses</code>	Structured vision output + <code>raw_model_output.json</code> .
<code>caf_core.creative_insights</code>	Operator-facing insight; <code>insight_ref</code> is used in <code>ideas_json.grounding_insight_ids</code> (prefix <code>ci_</code>).
<code>caf_core.creative_carousel_mimic_templates</code>	Stored <code>.hbs</code> source + file name for traceability.

Signal pack hints

`signal_pack_publication_hints` (via `buildCreationPack`) includes `top_performer_styling_cues` from `derived_globals.json.top_performer_styling_cues_v1` .

Planner weighting

When materializing ideas to planner rows, any idea whose `grounding_insight_ids` contains a ref starting with `ci_` gets a higher `past_performance` score (config

CREATIVE_INTEL_PLANNER_PAST_PERFORMANCE_BOOST).

Grounding uniqueness

POST /v1/signal-packs/.../ideas rejects duplicate **grounding_insight_ids** across ideas (each insight id may ground at most one idea).

Generation prompts

When **CREATIVE_INTEL_INJECT_IN_GENERATION=1** , a capped **creative style guidance** block is appended to the system prompt (and **creative_style_guidance** is available as a template placeholder but omitted from **{{creation_pack_json}}**).

Sample ingest

```
{
  "platform": "Instagram",
  "items": [
    {
      "source_url": "https://example.com/p/abc",
      "external_source_id": "abc",
      "media_type": "carousel",
      "media_urls": ["https://cdn.example.com/1.jpg",
"https://cdn.example.com/2.jpg"],
      "caption": "Hook text",
      "metrics": { "likes": 1200, "saves": 200 }
    }
  ],
  "selection_reason": "top_performer_scrape"
}
```

Limits

- Video temporal understanding requires a **direct downloadable** MP4 (or similar) and **ffmpeg** on the Core host (or frames are skipped).
- OpenAI vision needs **HTTPS** image URLs (Supabase **public_url** after upload works).
- Template mimicry v1 adjusts **palette variables only** — not full layout cloning.

Downstream: mimic lanes (July 2026)

When `visual_guidelines_pack_v1` entries include archived inspection media and Nemotron `mimic_evaluation` , grounded ideas can become mimic-family jobs (requires `MIMIC_IMAGE_ENABLED=1` on Core for pixel render).

Lane	Flow	Notes
Manual mimic carousel	FLOW_TOP_PERFORMER_MIMIC_CAROUSEL	TP reference frames; execution_mode: classic
New visual carousel	FLOW_VISUAL_FIRST_CAROUSEL	Not TP replication — execution_mode: new_visual, idea + BVS, empty reference_items
Why Mimic carousel	FLOW_WHY_MIMIC_CAROUSEL	SIL on mimic_v1.slide_intelligence; execution_mode: why_mimic
Mimic image	FLOW_TOP_PERFORMER_MIMIC_IMAGE	Single-frame image_full

Slide Intelligence (SIL) — stored on insights / `mimic_v1` for Why Mimic strategic copy and image prompts
(`migrations/071_inputs_evidence_insights_slide_intelligence.sql`).

Brand Visual System — `brand_bibles` → `generation_payload.bvs_v1` when `use_brand_visual_system` is set on the planned candidate. See `src/domain/brand-bible.ts` , `bvs-v1.ts` .

Doc	Content
CAF CURRENT STATE CONTEXT PACK.md	Repo-derived current truth (start here if stale)
MIMIC FLOWS COMPLETE GUIDE.md	Full mimic pipeline (modes, payloads, BVS, new visual, Why Mimic)
MIMIC IMAGE FLOWS.md	Operator quick reference
CAF INPUTS PIPELINE ROADMAP.md	Admin inputs → top-performer insight tiers